

Redes de Computadoras — TP Teórico: La Capa de Transporte

Problema 1

UDP (User Datagram Protocol, RFC 768) es un protocolo de transporte sin conexión y no confiable: envía datagramas sin establecer conexión previa, sin garantía de entrega, sin orden y sin control de flujo. Lo único que agrega sobre IP es la multiplexación por puertos y un checksum (opcional en IPv4). Su encabezado son 8 bytes:

Puerto origen (16 bits)	Puerto destino (16 bits)
Longitud UDP (16 bits)	Checksum UDP (16 bits)
Datos (carga útil)	

Ejemplos de uso: DNS, DHCP, TFTP, SNMP, streaming de audio y video, VoIP y juegos en línea, casos donde importa la velocidad y la baja latencia y se tolera alguna pérdida.

Problema 2

TCP es orientado a conexión, confiable (ACK y retransmisión), ordenado, con control de flujo y de congestión, full-dúplex; tiene más overhead y latencia. UDP es sin conexión, no confiable, sin orden ni control de flujo, con overhead mínimo y muy rápido.

Cuándo cada uno: TCP es preferible cuando importa que llegue todo y en orden (web, correo, transferencia de archivos). UDP es preferible en tiempo real (VoIP, video, juegos) y en consultas simples de pregunta y respuesta (DNS), donde una retransmisión llegaría tarde y molesta más que la propia pérdida.

Problema 3

En el three-way handshake se intercambian, en las opciones de los segmentos SYN: el número de secuencia inicial (ISN) de cada lado, el MSS (tamaño máximo de segmento), el tamaño de ventana y el window scaling, el uso de SACK (selective ACK) y de timestamps. Son los parámetros que ajustan cómo va a funcionar esa conexión.

Problema 4

Porque IP entrega paquetes de host a host, pero no tiene puertos: no sabe a qué proceso o aplicación entregar los datos dentro del host. UDP agrega justamente esa multiplexación y

demultiplexación por puertos, para que varios procesos usen la red a la vez, más un checksum opcional de los datos. Además los procesos de aplicación no acceden directamente a IP (es del kernel); UDP les da una interfaz de sockets simple. Sin un transporte no habría forma de dirigir el datagrama al proceso correcto.

Problema 5

Es que varios procesos o conexiones compartan la capa de red de un mismo host. En el emisor, la capa de transporte toma datos de varios sockets (cada uno identificado por su puerto) y los multiplexa hacia abajo, hacia una única capa de red. En el receptor demultiplexa: según el puerto destino (y en TCP, la cupla completa origen y destino) entrega cada segmento al socket o proceso que corresponde.

Proceso A (pto 1), Proceso B (pto 2), Proceso C (pto 3) → [Capa de transporte] → [Capa de red]
→ un solo canal

Problema 6

Elemento del protocolo	TCP	UDP
Establecimiento de conexiones	Sí	No
Particionamiento de mensajes largos en segmentos	Sí	No
Transferencia de segmentos	Sí	Sí
Numeración de los segmentos	Sí	No
Control de flujo de nivel de transporte	Sí	No
Multiplexación	Sí	Sí
Retransmisión debida a timeout	Sí	No
Resecuenciamiento de segmentos	Sí	No
Checksum de nivel de transporte	Sí	Sí (opcional en IPv4)

Problema 7

No. Una conexión TCP se identifica por la cupla de 4: IP origen, puerto origen, IP destino y puerto destino. Si los cuatro valores son iguales (mismas IPs y los mismos puertos p y q), sería exactamente la misma conexión, y no puede haber dos idénticas simultáneas. Para tener dos conexiones hace falta que difiera al menos uno de los cuatro, por ejemplo otro puerto en alguno de los extremos.

Problema 8

Ambos corren sobre IP, que es un servicio de red no orientado a conexión y no confiable (best-effort). TCP está diseñado para ofrecer, encima de ese servicio no confiable, un servicio confiable y orientado a conexión, agregando la confiabilidad que IP no da. UDP trabaja sobre el mismo servicio no confiable y no agrega confiabilidad: refleja el best-effort de IP tal cual.

Problema 9

No. Como es sin conexión y no garantiza orden ni entrega, no necesita numerar, reordenar ni detectar pérdidas: cada datagrama es independiente. Si una aplicación sobre UDP necesita orden, por ejemplo streaming con RTP, lo implementa ella misma por encima.

Problema 10

a. Sin conexión. UDP entrega datagramas sin establecer conexión previa.

Problema 11

Sí. TCP numera los bytes del flujo: el número de secuencia es la posición del primer byte del segmento. Sirven para ordenar segmentos que llegan desordenados, detectar pérdidas y duplicados, y confirmar la recepción, ya que los ACK indican el próximo byte esperado. Son la base de la entrega confiable y ordenada.

Problema 12

netstat muestra las conexiones de red activas (TCP y UDP), los puertos en escucha, sus estados, tablas de ruteo y estadísticas. Sirve para ver qué conexiones y puertos tiene abiertos un host y diagnosticar, por ejemplo qué servicios escuchan y con quién está conectado.

Errores en la salida:

- Conexión duplicada: la línea 200.11.163.35.110 con 200.5.114.77.3888 en ESTABLISHED aparece dos veces. Dos conexiones no pueden compartir la misma cupla de 4 y estar ambas ESTABLISHED: es imposible, una de las dos sobra.
- Dirección remota privada: 172.18.105.129 es una IP privada (RFC 1918). En un servidor con IP pública no debería aparecer como par remoto directo desde Internet, porque llegaría NATeada a una IP pública. Es sospechosa e inconsistente.

Problema 13

a) Criterio: se separan por la cupla de 4 (IPs y puertos de origen y destino). Hay dos conexiones:

- Conexión A: 10.1.0.1:1234 con 10.3.2.4:80. Paquetes 1, 2, 4, 6, 9, 11, 13, 14, 16, 19, 22.

- Conexión B: 3.14.15.92:654 con 2.71.82.81:82. Paquetes 3, 5, 7, 8, 10, 12, 15, 17, 18, 20, 21, 23.

b) Inicio y cierre de cada conexión:

- Conexión A, inicio (three-way handshake): paq 1 (SYN), 2 (SYN-ACK), 4 (ACK). Cierre: paq 16 (FIN-ACK del servidor), 19 (FIN-ACK del cliente), 22 (ACK final).
- Conexión B, inicio: paq 3 (SYN), 5 (SYN-ACK), 7 (ACK). Cierre: paq 18 (FIN), 20 (ACK), 21 (FIN), 23 (ACK).

c) Retransmisión: en la Conexión B, los paquetes 12 y 15 son idénticos (mismo origen y destino, Seq=302, longitud 1000). El original es el paq 12 y la retransmisión es el paq 15, que se reenvió porque todavía no había llegado el ACK que confirmara esos datos.

Problema 14

Criterio: la cupla de 4. Hay dos conexiones: los dos clientes se conectan al mismo servidor 157.92.23.3:801, pero son conexiones distintas porque difiere el extremo cliente (IP y puerto).

- Conexión 1: 157.92.75.5:1024 con 157.92.23.3:801. Paquetes 1, 2, 4, 7, 8, 10, 12, 13, 16, 17, 19, 20, 22, 23, 24, 25, 27, 28.
- Conexión 2: 190.30.132.239:3623 con 157.92.23.3:801. Paquetes 3, 5, 6, 9, 11, 14, 15, 18, 21, 26.

Observación: el paquete 20 figura con destino 157.92.23.3:80 (puerto 80 en lugar de 801). Tomado literalmente sería otra cupla y no pertenecería a la Conexión 1; es casi seguro un error de tipeo por 801.

Problema 15

Sí, es posible. Cada ventana abre su propia conexión TCP con un puerto de origen efímero distinto en la PC. Aunque el destino sea el mismo (168.83.72.5:80) y la IP origen la misma (157.92.27.33), como el puerto de origen difiere las cuplas de 4 son distintas, así que son dos conexiones separadas. Los paquetes de cada ventana se distinguen por ese puerto de origen del cliente (la cupla completa).

Problema 16

Falta el tercer segmento del handshake: el cliente responde con un ACK (seq 101, ack 201) confirmando el SYN del servidor. Con eso el three-way handshake queda completo, la conexión pasa a ESTABLISHED y empieza la transferencia de datos. Nota: el ACK del servidor debería ser 101, es decir el ISN del cliente más 1, porque el SYN consume un número de secuencia.

Problema 17

UDP. En tiempo real lo que importa es la baja latencia. La retransmisión de TCP llegaría tarde (un paquete de voz retransmitido ya no sirve) y su control de congestión y de orden agrega demoras. Es preferible perder algún paquete, que se escucha como un pequeño corte, antes que esperar retransmisiones. Por eso la telefonía sobre IP usa UDP, normalmente con RTP por encima.

Problema 18

Por la cupla de 4, en particular por el puerto de origen que A usó para cada conexión. A abrió cada una con un puerto efímero distinto; cuando B responde, el segmento trae ese puerto como puerto destino. A demultiplexa según la cupla completa (IPs y puertos) y así sabe a qué conexión o socket corresponde la respuesta.

Problema 19

La correcta es la c. SYN – SYN/ACK – ACK, que es el three-way handshake: el cliente manda SYN, el servidor responde SYN+ACK y el cliente confirma con ACK. Las demás no representan un establecimiento válido de TCP.

Problema 20

a) Elección de los puertos: el puerto destino es el well-known del servicio, 25 para SMTP (y 53 para el DNS de los frames 1 y 2). El puerto origen (1204) es un puerto efímero que el sistema operativo del cliente asigna dinámicamente a esa conexión.

b) Segmentos 3, 4 y 5: son el three-way handshake de TCP, previo a hablar SMTP:

- 3, cliente a servidor: SYN, SEQ=3405653374 (ISN del cliente).
- 4, servidor a cliente: SYN+ACK, SEQ=2978859689 (ISN del servidor), ACK=3405653375, que es el ISN del cliente más 1, confirmando su SYN.
- 5, cliente a servidor: ACK=2978859690, que es el ISN del servidor más 1, confirmando su SYN.

Los SEQ son los ISN (números iniciales, elegidos al azar) de cada lado; cada ACK es el ISN del otro más 1, porque el SYN consume un número de secuencia. Con esto queda establecida la conexión.

c) Líneas 1 y 2: son la consulta DNS. En el frame 1 el cliente (209.13.34.94) le pregunta a su servidor DNS (200.61.33.5) por el nombre mail.sinectis.com.ar (query, ID=93); en el frame 2 el DNS responde (STAT=OK) con la resolución del nombre a su IP. Es decir, antes de conectarse por SMTP, el cliente resuelve por DNS el nombre del servidor de correo.